

LAB 2 · PISO

VS Code에서 직접 작성·검증한 다음 장비로 확인한다.

작성일 2026. 09. 10. · 이해리

Vivado 실습

목차

프로젝트 시작

RTL·TB·XDC 직접 작성

VS Code 시뮬레이션과 수정

구현과 장치 기록

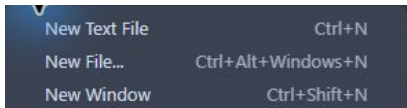
레포트 공통 예습 전체 목차

v2.0.1 템플릿을 새 이름으로 clone

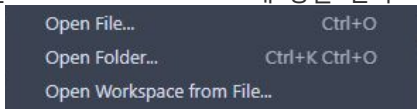
Windows PowerShell에서 한 줄씩 실행한다.

```
git clone --branch v2.0.1 `
https://github.com/Glaysia/fpga-lab-template.git `
lab2_05_piso
cd lab2_05_piso
git switch -c main
code LAB1.code-workspace
```

File → New Window



Ctrl+Shift+N 또는 File → New Window. 새 창을 연다.



File → Open Workspace from File...을 누른다.

workspace와 추천 확장

1. lab2_05_piso 폴더에서 LAB1.code-workspace 선택 → Open.
2. Explorer에 PROJECT 하나와 src.sim.constraints.tools가 보이는지 확인.
3. Extensions → @recommended를 검색.
4. slang.VaporView.vscode-pdf의 Install을 누른다.
5. Python과 Icarus 실행 파일은 별도로 설치되어 있어야 한다.

v2.0.1 편집기와 시뮬레이터

1. slang 확장·서버는 0.3.0을 기준으로 확인한다. 추천 목록은 버전을 고정하지 않는다.
2. 템플릿의 .slang/server.json은 src·sim 탐색용이다. 새 RTL은 simulation.json에도 직접 등록한다.
3. Ctrl+Shift+P → slang: Show Output. 서버 오류가 있으면 README의 버전 확인·재시작 절차를 따른다.
4. slang.path에 다른 사람의 설치 경로를 복사하지 않는다. 직접 지정할 때는 User 설정에 자신의 경로를 쓴다.
5. VS Code 02 Simulate는 Icarus, Vivado Run Behavioral Simulation은 XSim이다.

템플릿 도구 설치·오류 해결 안내

시뮬레이션 도구 확인

1. Terminal → Run Task... → 01 Check tools.
2. Git·Python·iverilog·vvp의 버전이 출력되는지 확인한다.
3. 도구를 찾지 못하면 설치 경로를 PATH에 추가하고 VS Code를 다시 연다.

```
git --version  
python --version  
iverilog -V  
vvp -V
```

빈 파일에서 시작

1. src/design.v를 우클릭 → Rename. 첫 RTL 파일명으로 바꾼다.
2. 추가 RTL은 src를 우클릭 → New File로 만든다.
3. sim/tb_design.v를 이번 TB 파일명으로 바꾼다.
4. constraints/pins.xdc를 이번 XDC 파일명으로 바꾼다.
5. 뒤의 코드 화면을 보며 작성하고 File → Save All.

이번 프로젝트에 작성할 파일

역할	경로
src	src/piso4.v
src	src/input_frontend.v
src	src/lab2_piso.v
sim	sim/tb_piso4.sv
constraints	constraints/lab2_piso.xdc

simulation.json의 세 항목

`sources`: 앞 표의 `src/` 파일 경로를 배열로 넣는다.

`testbench`: `sim/tb_piso4.sv`

`simulation_top`: `tb_piso4`

큰따옴표·대괄호·심표를 확인한다. `module` 이름에는 파일 확장자를 붙이지 않는다.

simulation.json · 1-9행

simulation.json를 열고 해당 구간을 직접 입력한다.

```
1 {  
2   "sources": [  
3     "src/piso4.v",  
4     "src/input_frontend.v",  
5     "src/lab2_piso.v"  
6   ],  
7   "testbench": "sim/tb_piso4.sv",  
8   "simulation_top": "tb_piso4"  
9 }
```

이번 회로의 관찰 방법

SW1..4(sw[7:4]): 병렬 데이터. SW8(sw[0])=1에서 버튼: load, 0에서 버튼: shift. LED[7:4]=저장값, LED[0]=직렬 출력.

입력 스위치를 먼저 정한 다음 버튼을 누른다.

네 비트를 저장하고 한 비트씩 읽는다

PISO는 Parallel In, Serial Out이다. load=1인 상승 에지에서 data_in의 네 비트를 value에 한 번에 저장한다.

serial_out은 항상 value[3]이다. 먼저 현재 MSB를 읽고, enable=1인 다음 상승 에지에서 {value[2:0], 0}으로 이동한다.

입력을 bit 3에 넣었던 앞 실험과 달리, 이번에는 저장값을 bit 3 방향으로 옮기고 bit 0을 0으로 채운다.

PISO의 제어 우선순위

rst	load	enable	다음 value
1	무관	무관	0
0	1	무관	data_in
0	0	1	{이전 value[2:0], 0}
0	0	0	유지

load와 enable이 모두 1이면 load가 우선이다. serial_out은 별도의 출력 레지스터가 아니라 저장값의 MSB를 연결한 선이다.

1010을 읽는 정확한 시점

관찰 시점	value	serial_out
load 직후, 첫 shift 전	1010	1
첫 shift 후	0100	0
둘째 shift 후	1000	1
셋째 shift 후	0000	0
넷째 shift 후	0000	0

네 비트 1,0,1,0을 읽으려면 각 shift 에지 전에 출력값을 기록한다.
shift 뒤에만 네 번 읽으면 첫 비트를 놓친다.

piso4.v · 1-14행

src/piso4.v를 열고 해당 구간을 직접 입력한다.

```
1  `timescale 1ns/1ps
2  module piso4 (
3      input wire clk, rst, load, enable,
4      input wire [3:0] data_in,
5      output wire serial_out,
6      output reg [3:0] value
7  );
8      assign serial_out = value[3];
9      always @(posedge clk) begin
10         if (rst) value <= 4'd0;
11         else if (load) value <= data_in;
12         else if (enable) value <= {value[2:0], 1'b0};
13     end
14 endmodule
```

input_frontend.v · 1-16행

src/input_frontend.v를 열고 해당 구간을 직접 입력한다.

```
1  timescale 1ns/1ps
2  // Main clock is 1 kHz: STABLE_CYCLES=20 means 20 ms.
3  // Set switches first, then press the button; the bus is not an atomic CDC.
4  module input_frontend #(parameter integer STABLE_CYCLES = 20) (
5      input wire clk, rst, button,
6      input wire [7:0] sw,
7      output wire reset,
8      output reg press,
9      output wire [7:0] switches
10 );
11     (* ASYNC_REG = "TRUE" *) reg [1:0] reset_pipe;
12     (* ASYNC_REG = "TRUE" *) reg button_meta, button_sync;
13     (* ASYNC_REG = "TRUE" *) reg [7:0] sw_meta, sw_sync;
14     localparam integer WIDTH = (STABLE_CYCLES < 2) ? 1 : $clog2(STABLE_CYCLES);
15     reg [WIDTH-1:0] count;
16     reg accepted;
```

input_frontend.v · 17-32행

src/input_frontend.v를 열고 해당 구간을 직접 입력한다.

```
17  assign reset = reset_pipe[1];
18  assign switches = sw_sync;
19  // Asynchronous assertion; release follows two main-clock edges.
20  always @(posedge clk or posedge rst) begin
21      if (rst) reset_pipe <= 2'b11;
22      else reset_pipe <= {reset_pipe[0], 1'b0};
23  end
24  always @(posedge clk or posedge reset) begin
25      if (reset) begin
26          button_meta <= 0; button_sync <= 0;
27          sw_meta <= 0; sw_sync <= 0;
28      end else begin
29          button_meta <= button; button_sync <= button_meta;
30          sw_meta <= sw; sw_sync <= sw_meta;
31      end
32  end
```

input_frontend.v · 33–46행

src/input_frontend.v를 열고 해당 구간을 직접 입력한다.

```
33     always @(posedge clk or posedge reset) begin
34         if (reset) begin
35             count <= 0; accepted <= 0; press <= 0;
36         end else begin
37             press <= 0;
38             if (button_sync == accepted) count <= 0;
39             else if (count == STABLE_CYCLES - 1) begin
40                 accepted <= button_sync;
41                 count <= 0;
42                 press <= button_sync;
43             end else count <= count + 1'b1;
44         end
45     end
46 endmodule
```

lab2_piso.v · 1-16행

src/lab2_piso.v를 열고 해당 구간을 직접 입력한다.

```
1  `timescale 1ns/1ps
2  // Combo II-DLD S75. Main clock B6 MUST be set to 1 kHz.
3  // K4: reset. N8: step button. DIPSW1..8 = sw[7]..sw[0].
4  module lab2_piso #(parameter integer STABLE_CYCLES = 20) (
5      input wire clk, rst, button,
6      input wire [7:0] sw,
7      output wire [7:0] led
8  );
9      wire reset, press;
10     wire [7:0] switches;
11     input_frontend #(.STABLE_CYCLES(STABLE_CYCLES)) inputs(
12         clk, rst, button, sw, reset, press, switches);
13     wire serial_out;
14     wire [3:0] value;
15     piso4 core(clk, reset, press && switches[0], press && !switches[0],
16         switches[7:4], serial_out, value);
```

lab2_piso.v · 17-18행

src/lab2_piso.v를 열고 해당 구간을 직접 입력한다.

```
17 |     assign led = {value, 3'b000, serial_out};  
18 | endmodule
```

TB는 모든 4비트 입력을 검사한다

word=0부터 15까지 반복한다. 각 word에서 load 우선, hold, 직렬 출력 4개, 네 번 shift 후 0 채움을 검사한다.

한 word당 7개 검사 $\times$ 16개 입력 + 처음 리셋 + 마지막 리셋 우선 = 114개다.

TB는 MSB를 먼저 비교한 뒤 step으로 이동한다. 같은 시각에 hold 검사와 첫 직렬 출력 검사가 함께 실행될 수 있다.

입력 A가 나오는 파형 구간

시각	관찰
616 ns	A 병렬 load 검사
626 ns	hold와 첫 serial bit=1 검사
636 ns	둘째 bit=0 검사
646 ns	셋째 bit=1 검사
656 ns	넷째 bit=0 검사
666 ns	네 번 shift 후 value=0

TB 전체 종료는 976 ns다. 위 시각은 반복문의 word=10 구간이며 첫 word=0 구간과 혼동하지 않는다.

tb_piso4.sv · 1-16행

sim/tb_piso4.sv를 열고 해당 구간을 직접 입력한다.

```
1 timescale 1ns/1ps
2 module tb_piso4;
3 reg clk=0, rst=1;
4 integer checks=0;
5 always #5 clk=~clk;
6 task step; begin @(posedge clk); #1; end endtask
7 task check(input condition, input [8*100-1:0] description);
8 begin
9     checks=checks+1;
10    if (condition !== 1'b1) begin
11        $display("LAB2_FAIL %0s time=%0t",description,$time);
12        $fatal(1,"check failed");
13    end
14 end
15 endtask
16 task finish;
```

tb_piso4.sv · 17-32행

sim/tb_piso4.sv를 열고 해당 구간을 직접 입력한다.

```
17 begin
18     $display("LAB2_PASS piso4 checks=%0d",checks);
19     $finish;
20 end
21 endtask
22 initial begin $dumpfile("wave.vcd"); $dumpvars(0,tb_piso4); end
23 initial begin #100000; $fatal(1,"watchdog timeout"); end
24 reg load=0, enable=0;
25 reg [3:0] data_in=0;
26 wire serial_out;
27 wire [3:0] value;
28 piso4 dut(clk,rst,load,enable,data_in,serial_out,value);
29 integer word, bit_number;
30 initial begin
31     step; check(value==0,"reset"); rst=0;
32     for(word=0;word<16;word=word+1) begin
```

tb_piso4.sv · 33-46행

sim/tb_piso4.sv를 열고 해당 구간을 직접 입력한다.

```
33      data_in=word; load=1; enable=1; step;
34      check(value===word,"load beats shift");
35      load=0; enable=0; step; check(value===word,"hold");
36      enable=1;
37      for(bit_number=3;bit_number>=0;bit_number=bit_number-1) begin
38          check(serial_out===((word>>bit_number)&1),"MSB first before edge");
39          step;
40      end
41      check(value===0,"four shifts zero fill");
42  end
43  rst=1; load=1; data_in=15; step;
44  check(value===0,"reset beats load"); finish;
45 end
46 endmodule
```

XDC를 직접 작성한다

1. constraints 폴더의 XDC를 열고 전체 내용을 입력한다.
2. get_ports의 이름과 비트 번호를 board top 선언과 대조한다.
3. 같은 핀을 서로 다른 포트에 중복 지정하지 않는다.
4. File → Save All. Icarus 기능 시뮬레이션은 XDC를 사용하지 않는다.

lab2_piso.xdc · 1-16행

constraints/lab2_piso.xdc를 열고 해당 구간을 직접 입력한다.

```
1  # Combo II-DLD S75 / xc7s75fgga484-1 / main clock 1 kHz.
2  # Pin source and array polarity: evidence/board-pin-provenance.json.
3  set_property PACKAGE_PIN B6 [get_ports {clk}]
4  set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
5  set_property PACKAGE_PIN K4 [get_ports {rst}]
6  set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
7  set_property PACKAGE_PIN N8 [get_ports {button}]
8  set_property IOSTANDARD LVCMOS33 [get_ports {button}]
9  set_property PACKAGE_PIN U4 [get_ports {sw[0]}]
10 set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
11 set_property PACKAGE_PIN V4 [get_ports {sw[1]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
13 set_property PACKAGE_PIN W1 [get_ports {sw[2]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
15 set_property PACKAGE_PIN W4 [get_ports {sw[3]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
```

lab2_piso.xdc • 17-32행

constraints/lab2_piso.xdc를 열고 해당 구간을 직접 입력한다.

```
17 set_property PACKAGE_PIN T1 [get_ports {sw[4]}]
18 set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
19 set_property PACKAGE_PIN U2 [get_ports {sw[5]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
21 set_property PACKAGE_PIN W3 [get_ports {sw[6]}]
22 set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
23 set_property PACKAGE_PIN Y1 [get_ports {sw[7]}]
24 set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
25 set_property PACKAGE_PIN N5 [get_ports {led[0]}]
26 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
27 set_property PACKAGE_PIN M1 [get_ports {led[1]}]
28 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
29 set_property PACKAGE_PIN M3 [get_ports {led[2]}]
30 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
31 set_property PACKAGE_PIN M7 [get_ports {led[3]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
```

lab2_piso.xdc · 33-42행

constraints/lab2_piso.xdc를 열고 해당 구간을 직접 입력한다.

```
33 set_property PACKAGE_PIN N7 [get_ports {led[4]}]
34 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
35 set_property PACKAGE_PIN M2 [get_ports {led[5]}]
36 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
37 set_property PACKAGE_PIN M4 [get_ports {led[6]}]
38 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
39 set_property PACKAGE_PIN L4 [get_ports {led[7]}]
40 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
41 create_clock -name trainer_1khz -period 1000000.000 [get_ports clk]
42 set_false_path -from [get_ports {rst button sw[*]}]
```

VS Code에서 먼저 시뮬레이션

1. File → Save All.
2. Terminal → Run Task... → 02 Simulate.
3. 첫 오류가 있으면 파일 경로·행 번호와 이전 행의 세미콜론부터 확인.
4. LAB2_PASS 또는 LAB2_INTEGRATED_PASS와 검사 수·종료 시각 확인.
5. Terminal → Run Task... → 03 Open waveform.

파형에서 확인할 신호

1. clk·rst·제어 입력·저장값·출력을 파형에 추가한다.
2. 상승 에지 직전 입력과 에지 뒤 상태를 비교한다.
3. 리셋 우선·enable=0 유지·마지막 검사 시각을 확인한다.
4. 출력만 본 화면과 전체 파형을 각각 저장한다.
5. 새 실행 로그와 파형인지 확인한다. 이전 PASS를 재사용하지 않는다.

정상 실행의 기준

```
LAB2_PASS piso4 checks=114
```

Icarus 종료 시각: 976ns. 로그가 ps로 표시되면 1,000으로 나눠 ns와 비교한다.

이 수치는 교안의 입력 순서·TB·파라미터에 대한 기준이다.

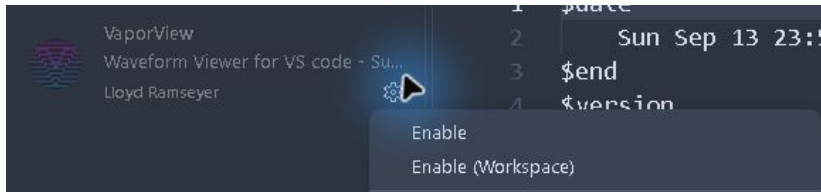
검사 수만 보지 말고 예상 상태와 실제 파형이 일치하는 이유를 설명한다.

PISO 자기검사 통과

```
VCD info: dumpfile wave.vcd opened for output.  
LAB2 PASS piso4 checks=114
```

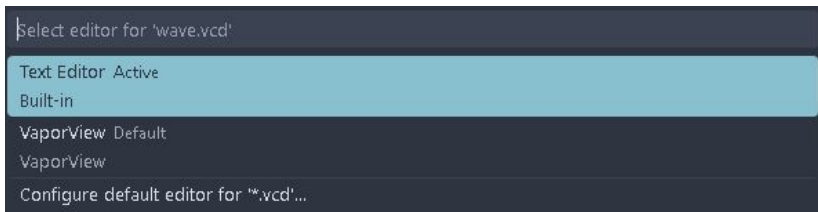
Ctrl+Shift+B 또는 Terminal → Run Task... → 02 Simulate. LAB2_PASS piso4 checks=114를 확인한다. 바로 다음 종료 로그는 976000 ps = 976 ns다.

설치된 파형 확장 활성화



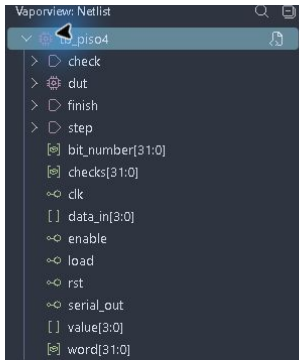
Extensions → @installed vaporview. 확장이 꺼져 있으면 톱니바퀴 → Enable (Workspace). 설치되지 않았다면 앞의 추천 확장 절차로 설치한다.

VCD를 VaporView로 열기



Explorer → build/sim/wave.vcd. 텍스트로 열리면 파일 탭 우클릭 → Reopen Editor With... → VaporView.

일곱 신호를 차례로 추가



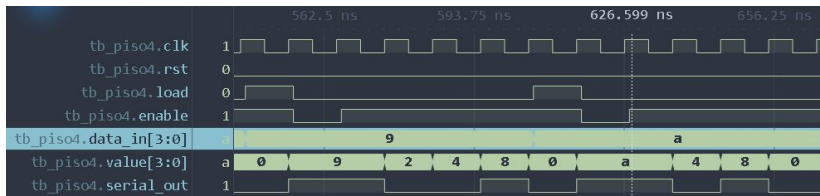
Netlist View → tb_piso4 펼치기. clk, rst, load, enable, data_in, value, serial_out을 차례로 더블클릭한다.

0-976 ns 전체 실행 확인



Zoom Fit으로 전체 구간을 맞춘다. 오른쪽 아래 Time Units → ns. 신호 이름이 잘리면 이름 열 경계를 오른쪽으로 끈다. data_in의 0부터 F까지 16개 입력을 확인한다.

A 입력의 로드·유지·시프트



600–670 ns 부근을 확대한다. 615 ns 에지에 A가 저장되고 625 ns 에지에는 유지된다. 635·645·655·665 ns 에지 뒤 value는 4·8·0·0이다. 화면의 버스 값은 16진수다.

파형과 TB 검사 시각 연결

TB의 step은 상승 에지 뒤 1 ns를 기다린다. 저장은 615 ns, load 검사는 616 ns다. 626 ns에는 hold와 첫 직렬 비트 1을 검사한다.

이후 636·646·656 ns에 읽는 serial_out은 0·1·0이다. 각 비트를 읽고 다음 step을 실행하므로 shift 뒤에만 읽는 방식과 구분한다.

VaporView 화면의 커서는 626.599 ns다. 이때 value=A, serial_out=1이다. TB 비교 시각 자체가 커서 시각인 것은 아니다.

수정 실험 · 실패와 복구

1. 정상 로그와 파형을 먼저 보관한다.
2. MSB 대신 LSB 출력하기 TB 기대값은 유지한다.
3. Save All → 02 Simulate. LAB2_FAIL의 검사 이름·시각을 읽는다.
4. 어떤 입력과 이전 상태에서 불일치가 발생했는지 계산한다.
5. RTL을 복구·저장·재실행하고 전체 PASS를 확인한다.

실험 전 레포트

1. 동작 표·상태 전이·타이밍을 예상한다.
2. RTL·TB·XDC·설정의 역할을 설명한다.
3. 정상/변경/복구 3행 비교표와 실행 로그를 넣는다.
4. VS Code 파형을 확대해 입력·이전 상태·다음 상태를 설명한다.
5. 장비 클록·핀·예상 LED 동작을 정리한 뒤 Vivado로 진행한다.

PISO 파형·수정 실험 해설

파형에 `clk·rst·load·enable·data_in·value·serial_out`을 추가한다.
`value`는 2진수로 표시하면 비트 이동을 읽기 쉽다.

`assign serial_out = value[3]`을 `value[0]`으로 바꾸면 `word=1`의 첫 출력 검사인 86 ns에서 실패한다. 기대값은 `MSB=0`, 변경 회로의 출력은 `LSB=1`이다.

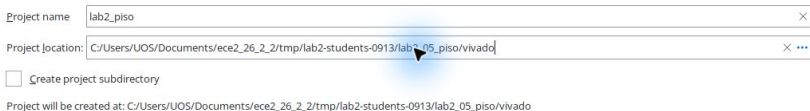
TB를 바꾸지 않고 RTL을 원복한다. 114개 PASS와 976 ns 종료를 확인하고 정상·변경·원복의 출력 순서를 비교한다.

Vivado에서 같은 PISO 원본 연결

VS Code 정상·변경·원복 검사와 실험 전 레포트를 마친 뒤 시작한다.

설계 Top은 lab2_piso, 시뮬레이션 Top은 tb_piso4다. 두 모듈의 용도를 구분한다.

프로젝트 이름과 위치



Project name: lab2_piso

Project location: C:/Users/UOS/Documents/ece2_26_2/tmp/lab2-students-0913/lab2_05_piso/vivado

☐ Create project subdirectory

Project will be created at: C:/Users/UOS/Documents/ece2_26_2/tmp/lab2-students-0913/lab2_05_piso/vivado

File → Project → New → Next. 이름은 lab2_piso, 위치는 자신의 lab2_05_piso/vivado. Create project subdirectory 해제 → Next.

RTL 프로젝트 선택

- ☒ RTL Project
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL
- ☒ Do not specify sources at this time
- ☐ Project is an extensible Vitis platform

RTL Project와 Do not specify sources at this time을 선택한다. Next.

정확한 부품 선택

Parts

Boards

Q

Q· xc7s75fgga484-1

ⓧ

▼

(3 matches)

Part	I/O Pin Count	Available IOBs	LUT Elements	FlipFlops	Block RAMs
xc7s75fgga484-1	484	338	48000	96000	90
xc7s75fgga484-1IL	484	338	48000	96000	90
xc7s75fgga484-1Q	484	338	48000	96000	90

Parts 검색란에 xc7s75fgga484-1을 입력한다. 같은 행 선택 → Next.
-1IL·-1Q와 구분한다.

생성 요약 확인

New Project Summary

- A new RTL project named 'lab2_piso' will be created.
- The default part and product family for the new project:
Default Part: xc7s75fgga484-1
Family: Spartan-7
Package: fgga484
Speed Grade: -1

lab2_piso·Spartan-7·fgga484·-1을 확인하고 Finish. 이전 프로젝트가 열려 있으면 저장 상태 확인 후 닫는다.

빈 프로젝트 확인

Settings [Edit](#)

Project name	lab2_piso
Project location	C:/Users/UOS/Documents/ece2_26_2_2/tmp/lab2-students-0913/lab2_05_piso/vivado
Product family	Spartan-7
Project part	xc7s75fpga484-1
Top module name	Not defined
Target language	Verilog
Simulator language	Mixed
Target Simulator	Vivado Simulator

Project Summary에서 이름·위치·부품을 확인한다. 아직 소스를 넣지 않았으므로 Top module은 Not defined다.

설계 RTL 세 파일 선택

Look in

src

- design.v
- input_frontend.v
- lab2_piso.v
- piso4.v

Add Sources → Add or create design sources → Next → Add Files. src
에서 input_frontend.v·lab2_piso.v·piso4.v를 선택하고 OK.

원본 RTL 참조

	Index	Name	Library	Version	Location
●	1	input_frontend.v	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/tmp/lab
●	2	lab2_piso.v	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/tmp/lab
●	3	piso4.v	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/tmp/lab

☒ Scan and add RTL include files into project

☐ Copy sources into project

세 파일을 확인한다. Copy sources into project 해제 → Finish. VS Code에서 편집한 원본을 계속 참조한다.

테스트벤치 등록

Specify simulation set: sim_1

+	-	↑	↓			
	Index	Name	Library	Version	Location	
●	1	tb_piso4.sv	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/tmp/lab2-stu	

- ☒ Scan and add RTL include files into project
- ☐ Copy sources into project
- ☐ Add sources from subdirectories
- ☒ Include all design sources for simulation

Add Sources → Add or create simulation sources → Next → Add Files → sim/tb_piso4.sv. sim_1과 Include all design sources 유지, Copy sources 해제 → Finish.

XDC 원본 참조

Specify constraint set: ⌵ constrs_1 (active) ⌵

+ - ↑ ↓	
Constraint File	Location
lab2_piso.xdc	C:\Users\UOS\Documents\ece2_26_2\tmp\lab2-students-0913\lab2_05_piso\constraints

☐ Copy constraints files into project

Add Sources → Add or create constraints → Next → Add Files → constraints/lab2_piso.xdc. Copy constraints files into project 해제 → Finish.

테스트벤치를 Top으로



Simulation Sources → sim_1을 펼친다. tb_piso4 우클릭 → Set as Top.
보드용 lab2_piso와 혼동하지 않는다.

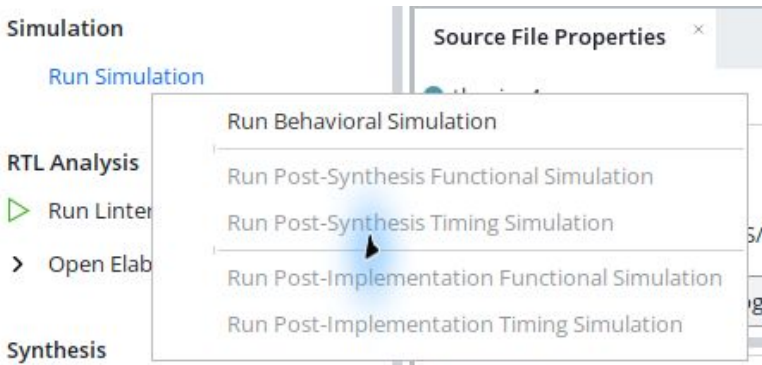
서로 다른 Top 확인

The screenshot displays the Vivado IDE interface. On the left, the 'Simulation Sources' tree is expanded, showing 'sim_1 (2) (active)' with two sub-items: 'tb_piso4 (tb_piso4.sv) (1)' and 'lab2_piso (lab2_piso.v) (2)'. The 'tb_piso4' item is highlighted with a blue selection bar. Below the tree, the 'Hierarchy' tab is active, showing the project's component hierarchy. On the right, the 'Project Summary' panel lists key project information:

Project name	lab2_piso
Project location	C:/Users/UOS/Documents
Product family	Spartan-7
Project part	xc7s75fpga484-1
Top module name	lab2_piso

tb_piso4가 굵게 표시된다. Project Summary의 설계 Top은 lab2_piso다.
이 상태로 시뮬레이션한다.

XSim 실행



왼쪽 Run Simulation → Run Behavioral Simulation. 컴파일과 elaboration이 끝날 때까지 기다린다.

XSim 114개 검사 통과

```
LAB2_PASS piso4 checks=114
```

```
$finish called at time : 976 ns : File
```

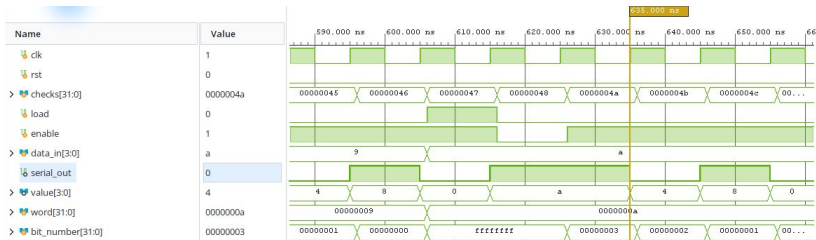
Tcl Console에서 LAB2_PASS piso4 checks=114와 \$finish 976 ns를 확인한다. 1000 ns 실행 요청보다 먼저 TB가 종료되는 것은 정상이다.

전체 파형 확인



파형 탭 더블클릭 → Zoom Fit. data_in이 0부터 F까지 바뀌며 모든 4비트 입력을 검사한다. Icarus와 같은 RTL·TB로 얻은 결과다.

A 입력을 직렬로 보내기



A 구간을 클릭한 뒤 Zoom In으로 확대한다. 615 ns에 A를 로드하고 625 ns에는 유지한다. 635·645·655 ns에 value가 4·8·0이 되며 MSB가 1·0·1·0 순서로 관찰된다.

LED 핀 확인

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	
v All ports (19)							
led (8)	OUT			✓	35	LVC MOS33*	
led[7]	OUT		L4	✓	35	LVC MOS33*	
led[6]	OUT		M4	✓	35	LVC MOS33*	
led[5]	OUT		M2	✓	35	LVC MOS33*	
led[4]	OUT		N7	✓	35	LVC MOS33*	
led[3]	OUT		M7	✓	35	LVC MOS33*	
led[2]	OUT		M3	✓	35	LVC MOS33*	
led[1]	OUT		M1	✓	35	LVC MOS33*	
led[0]	OUT		N5	✓	35	LVC MOS33*	

Open Elaborated Design → OK. 오른쪽 위 I/O Planning → I/O Ports 탭 더블클릭. led를 펼쳐 Package Pin·LVC MOS33을 XDC와 대조한다.

스위치 핀 확인

 SW (8)	IN			✓	34	LVC MOS33*	▼	
 sw[7]	IN		Y1	▼	✓	34	LVC MOS33*	▼
 sw[6]	IN		W3	▼	✓	34	LVC MOS33*	▼
 sw[5]	IN		U2	▼	✓	34	LVC MOS33*	▼
 sw[4]	IN		T1	▼	✓	34	LVC MOS33*	▼
 sw[3]	IN		W4	▼	✓	34	LVC MOS33*	▼
 sw[2]	IN		W1	▼	✓	34	LVC MOS33*	▼
 sw[1]	IN		V4	▼	✓	34	LVC MOS33*	▼
 sw[0]	IN		U4	▼	✓	34	LVC MOS33*	▼

sw를 펼쳐 여덟 핀을 확인한다. DIP SW1-SW4는 sw[7:4] 병렬 입력, SW8은 sw[0] load다. 스위치 번호와 비트 번호를 구분한다.

버튼·클록·리셋 핀 확인

Scalar ports (3)							
button	IN		N8	✓	✓	35	LVC MOS33* ✓
clk	IN		B6	✓	✓	36	LVC MOS33* ✓
rst	IN		K4	✓	✓	35	LVC MOS33* ✓

Scalar ports를 펼친다. button=N8, clk=B6, rst=K4다. 19개 포트 모두 LVC MOS33이다. B6 주 클록은 1 kHz로 설정한다.

합성 실행

Launch directory

 <Default Launch Directory>



Options



Launch runs on local host:

Number of jobs:

4



Generate scripts only

왼쪽 Run Synthesis. Launch runs on local host와 jobs를 확인하고 OK.
이 실행은 jobs=4다.

합성 완료

 Synthesis successfully completed.

Next

- ☒ Run Implementation
- ☐ Open Synthesized Design
- ☐ View Reports

Synthesis successfully completed를 확인한다. Run Implementation 선택
→ OK.

구현 실행

Launch directory

 <Default Launch Directory>



Options



Launch runs on local host:

Number of jobs:

4



Generate scripts only

Launch runs on local host와 jobs를 확인하고 OK. 구현 도중에는 RTL·XDC를 수정하지 않는다.

구현 완료

 Implementation successfully completed.

Next

- ☐ Open Implemented Design
- ☒  Generate Bitstream
- ☐ View Reports

Implementation successfully completed를 확인한다. Generate Bitstream 선택 → OK.

비트스트림 생성 실행

Launch directory

 <Default Launch Directory>



Options



Launch runs on local host:

Number of jobs:

4



Generate scripts only

Launch runs on local host와 jobs를 확인하고 OK. write_bitstream 완료까지 기다린다.

비트스트림 생성 완료

 Bitstream Generation successfully completed.

Next

- ☒ Open Implemented Design
- ☐ View Reports
- ☐ Open Hardware Manager
- ☐ Generate Memory Configuration File

Bitstream Generation successfully completed 확인. Open Implemented Design → OK. 이전 Elaborated Design을 닫을지 물으면 Yes.

타이밍 확인

Design Timing Summary

Setup		Hold	
Worst Negative Slack (WNS)	999997.500 ns	Worst Hold Slack (WHS)	0.121 ns
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns
Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	42	Total Number of Endpoints	42

Timing → Design Timing Summary. Setup·Hold의 slack과 Number of Failing Endpoints를 확인한다.

검사 결과와 생성 파일

XSim: LAB2_PASS piso4 checks=114, 976 ns 종료. Icarus와 같은 TB와 원복한 RTL을 사용했다.

생성 파일: vivado/lab2_piso.runs/impl_1/lab2_piso.bit.

GUI 합성·구현·bit 생성까지 확인했다. 장치 기록과 보드 사진·영상은 학생이 실습 장비에서 수행한다.

타이밍과 경고 해석

WNS=999997.500 ns, WHS=0.121 ns. Setup·Hold 실패 endpoint는 각각 0이다. 주 클럭 제약은 1,000,000 ns다.

TIMING-18: 외부 입출력 지연 관련 경고다. 비동기 입력에는 false path를 적용했고 LED 출력 지연은 지정하지 않았다. 내부 타이밍 통과가 모든 외부 입출력 타이밍을 보장하지는 않는다.

CFGBVS-1: 설정 बैं크 전압 속성이 없다. 보드 회로도와 설정 전압을 확인하여 값을 정한다. 경고만 없애려고 임의의 전압을 입력하지 않는다.

장치에서 동작 확인

1. 보드 전원·JTAG·주 클럭 1 kHz 설정을 확인한다.
2. Open Hardware Manager → Open target → Auto Connect.
3. Program Device에서 방금 생성한 bit 파일을 선택한다.
4. 초기화 뒤 입력·버튼·출력을 관찰하고 예상과 비교한다.
5. 사진·영상과 관찰 결과를 실험 후 레포트에 기록한다.

PIS0 보드 조작 순서

스위치 / 버튼	LED[7:0]
SW1..4=1010, SW8=1, N8	A1
SW8=0, N8 한 번	40
N8 한 번 더	81
N8 한 번 더	00
N8 한 번 더	00

상위 네 LED는 value, LED0은 직렬 출력이다. load 후 shift 전에 LED0=1을 먼저 읽고 이후 0,1,0을 관찰한다.

실험 후 레포트와 GitHub

1. 도구 버전·source commit·XDC·bit 해시·타이밍과 경고를 적는다.
2. 실제 기록 화면과 보드 전체 사진을 포함한다.
3. 버튼 누름과 출력 변화가 함께 보이는 영상을 찍는다.
4. 통합판은 8개 모드 번호·LCD 이름·회로 출력을 보여 준다.
5. 소스·실험 전후 레포트·사진·영상 링크를 GitHub에서 연결한다.